CSEN 11 Advanced Programming

Arrays – Simplest data structure

Static variables

- Variable definition modifier keyword: **static**
- Static variables are only allocated once
- Static variables are not deallocated until program ends

```
int keep_count() {
    static int count = 0;
    // count will remain allocated and keep its value
    count++;
    return count;
}
```

Static variables - Example

```c
#include <stdio.h>
void display();

int main()
{
    display();
    display();
}
void display()
{
    static int c = 1;
    c += 5;
    printf("%d  ",c);
}
```

Output:

```
6  11
```

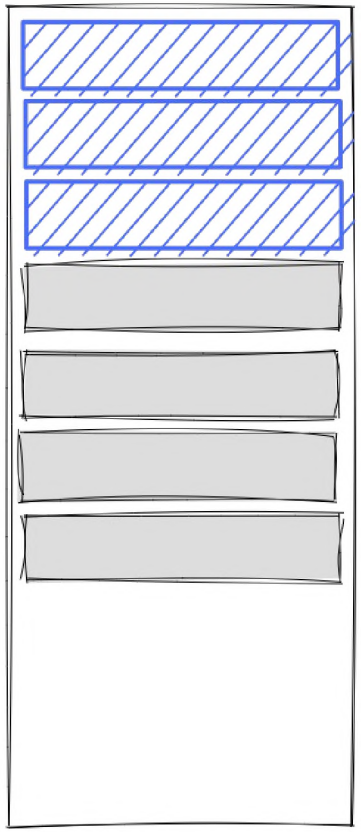
Arrays

- Programs often need to store **many values of the same type**
- Declaring separate variables is messy and unscalable

```
int score1, score2, score3, score4, score5;
```

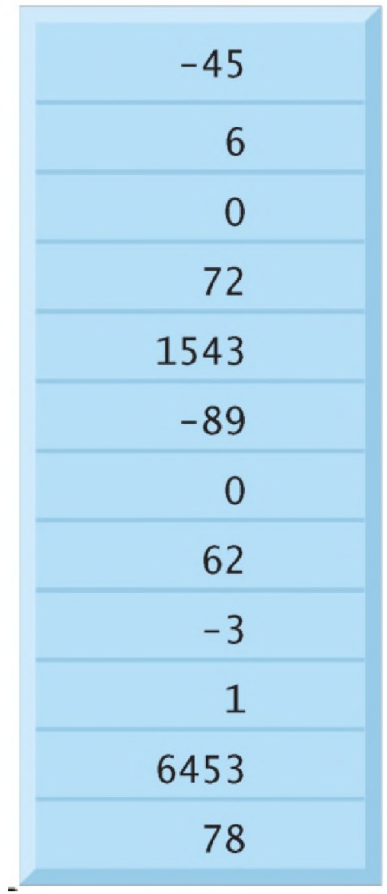
- An **array** stores multiple values **under one name**
- All elements:
  - Have the **same data type**
  - Are stored in **contiguous memory**
  - Are accessed using an **index**

```
int scores[5];
```



Arrays

- An array is a collection of 2+ adjacent memory cells
- A memory cells are called array elements
- An array is associated with a symbolic name
- Can manipulate and use
  - Individual elements OR
  - ‘list’ as one entity.



Declaring Arrays

- To declare an array, you need:
  - name
  - data type of the elements
  - size of the a

General Format for declaring arrays

```
<data type>  <variable> [<size>];
```

Example

int x[5];

- Ar

name x

index	0	1	2	3	4
Element value	13	28	27	5	71

Declaring Arrays - Examples

- General Format for declaring arrays

```
<data type>  <variable> [<size>];
```

- Declaration
  - Declaring the array → allocates memory
  - Static entity - same size throughout program
- Examples

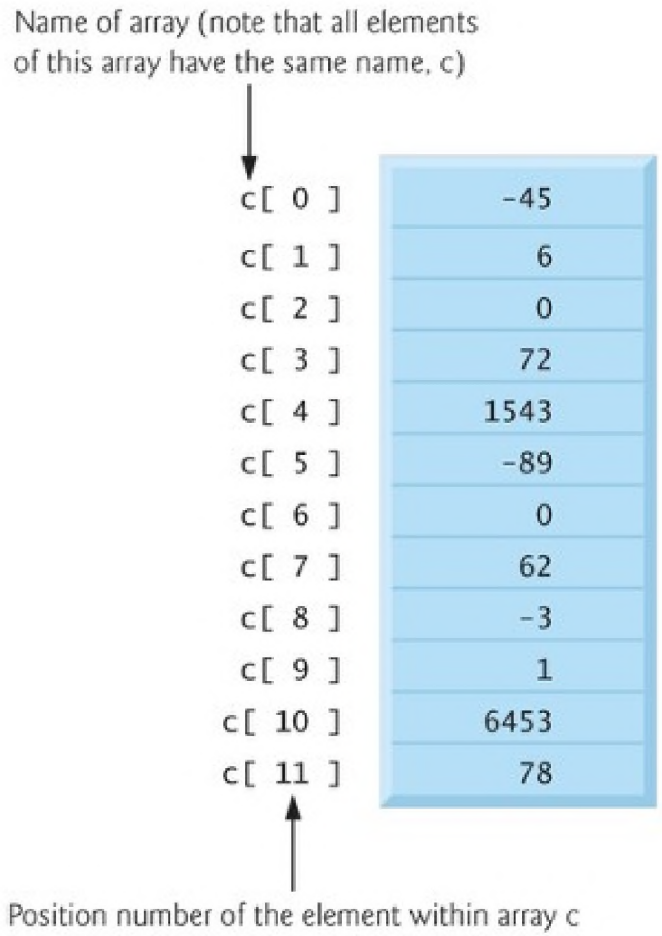
```
int c[12];
int scores[300];
float weight[3284];
char alphabet[26];
```

Type is int.
Name is c.

Referencing Arrays

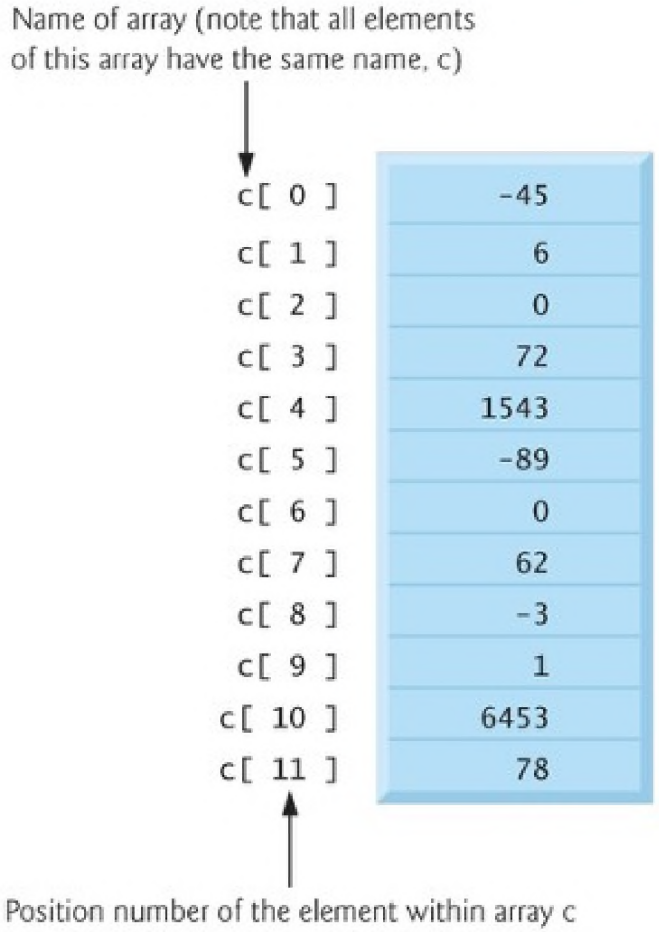
- To reference the array
  - Use the name of the array

Example: c



Referencing Arrays

- To reference each individual element
  - Use the name of the array and the index of the element
    - The index is given by the subscript in brackets
- Subscripts must be within the right range
  - If the array has size SIZE
    - Subscript range is zero to SIZE – 1
  - Using an out-of-range subscript may
    - Produce wrong results or
    - Crash the program → run-time error



Referencing Arrays

- Subscripts are integers
  - Constants, variables, or expressions

Examples:

```
x[4] = x[5];
x[i] = 0;
b = x[i];
x[i + j] = a;
```

Initializing Arrays

- Arrays can be initialized at declaration

```
int scores[3] = {76, 98, 83};
```

  - Size cannot be variable or named constant
  - Equivalent to

```
int scores[3];
scores[0] = 76;
scores[1] = 98;
scores[2] = 83;
```

  - Fills from beginning
  - Fills 'rest' with zero of array base type
    - Declaration

```
int scores[5] = {76, 98, 83}
```

    - Performs initialization

```
scores[0] = 76;
scores[1] = 98;
scores[2] = 83;
scores[3] = 0;
scores[4] = 0;
```

- If array size is left out
  - Declares array with size required based on number of initialization values
  - Example:

```
int scores[] = {76, 98, 83}
```

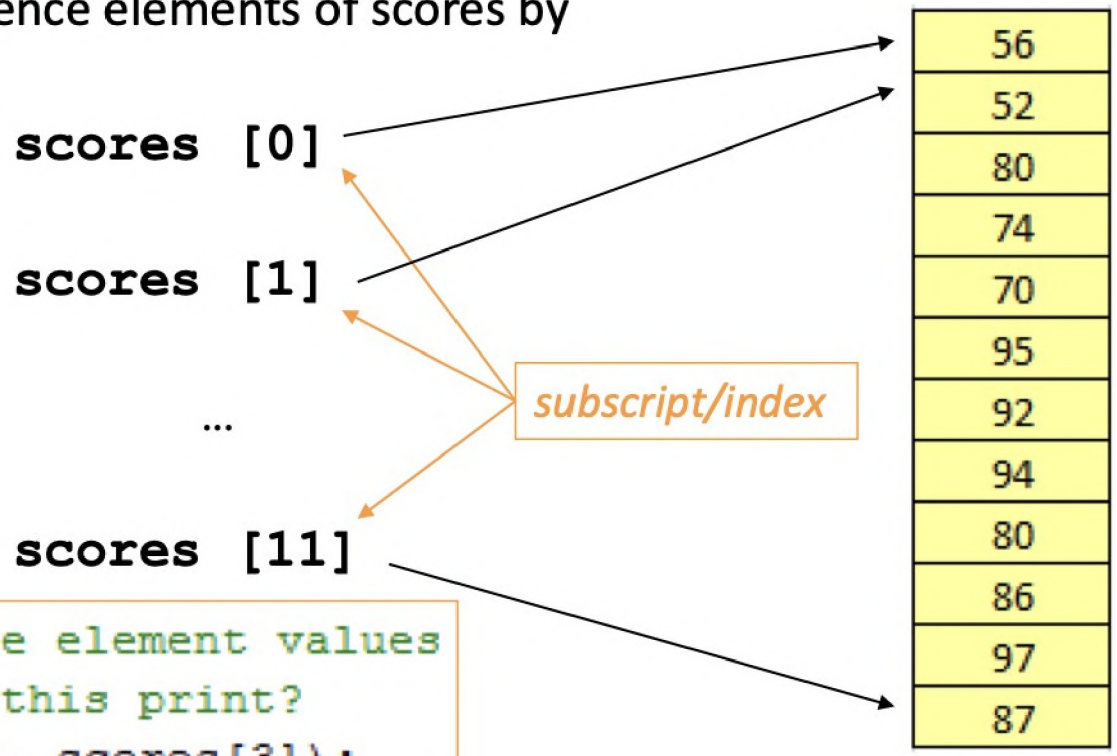
    - Allocates array scores with size of 3

Accessing Arrays

```
int scores[12];
```

- We reference elements of scores by

- Note two uses of brackets:
  - In declaration, specifies SIZE of array
  - Anywhere else, specifies a subscript/index




```
// Given these element values
// What does this print?
printf("%d\n", scores[3]);
```

For loop for Arrays

```
int scoreSub;
// Print forward
for (scoreSub = 0; scoreSub < 12; scoreSub++)
   printf("Score %d is %d\n", scoreSub+1,
            scores[scoreSub]);
// Print backward, in reverse
for (scoreSub = 11; scoreSub >= 0; scoreSub--)
   printf("Score %d is %d\n", scoreSub+1,
            scores[scoreSub]);
```

56
52
80
74
70
95
92
94
80
86
97
87

```
Score 1 is 56
Score 2 is 52
Score 3 is 80
Score 4 is 74
...
Score 12 is 87
```

```
Score 12 is 87
Score 11 is 97
Score 10 is 86
Score 9 is 80
...
Score 1 is 56
```

Multidimensional Arrays

- Arrays with 2 or more dimensions
- Used to represent
  - Tables
  - Matrices
  - Any two-dimensional objects

Requires two subscripts – row and column

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Column index
Row index
Array name

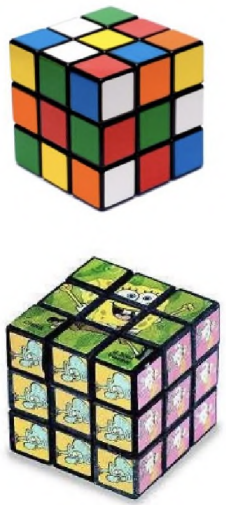
Multidimensional Arrays

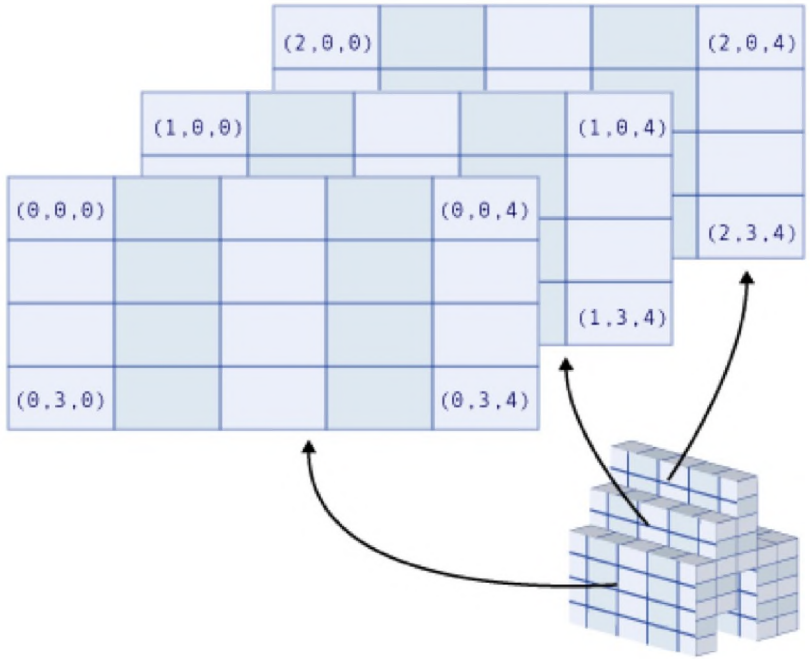
- Declaration
  - name,
  - type,
  - size of each dimension
- Examples

double matrix[20][20];

int multi[10][10][5];

```
int cube[3][3][3];
```





Multidimensional Arrays

- Initialization
  - Static: Values are grouped by dimension!

```
int   matrix[3][3] = {{0, 0, 0}, {0, 0, 0}, {0, 0, 0}};

int   matrix[3][3][3] = {{{0, 0, 0}, {0, 0, 0}, {0, 0, 0}},
                         {{0, 0, 0}, {0, 0, 0}, {0, 0, 0}},
                         {{0, 0, 0}, {0, 0, 0}, {0, 0, 0}}};
```

Multidimensional Arrays

- Initialization
  - Dynamic: Use nested loops!
  - Example

```
int nums[4][5] = { {10,  6, -7, 13, 28},
                   {10,  5, 44,  8},
                   {33, 20,  1,  0, 14},
                   { 2, 66, 25, 37,  1}
                 }
```

```
…
int   nums[4][5];
int sum;
…
for (i = 0; i < 4; i++)
    for (j = 0; j < 5; j++)
        sum += matrix[i][j];
```

Functions and Arrays

- The name of an array represents its address in memory
- Passing an array as an argument to a function is done by <u>reference</u>
  - Example:

```
int  array[50];
int        value;
…
return_value = search (array, value);
```

Function and Arrays

- Functions receiving arrays as arguments need to specify all but the first dimension size
- Example with a 1D array:

```
int search (int data [ ], int size)
{
    …
}
```

```c
#include <stdio.h>

void printArray(int data[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", data[i]);
    }
}

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};

    printArray(numbers, 5);
    return 0;
}
```

Function and Arrays

- Example with a 2D array:

```
int
search (int array [ ][NCOLS], int value)
{
    ...
}
```